Практическая работа №1. Разработка перечня артефактов и протоколов проекта

3.1. Выбор проекта

Выбираю учебный проект - **«TeamTask»: система управления задачами для малой команды**. Это веб-приложение для небольшой команды (примерно 3-10 человек), где можно вести задачи, назначать исполнителей, ставить сроки и отслеживать статус работы. Проект не из одного файла: в нём есть требования, проектирование, код, тестирование, развёртывание и сопровождение. Разработка идёт на **Python** (например, backend на Django или FastAPI), поэтому некоторые артефакты и протоколы будут связаны с этим языком.

3.2. Описание проекта

Название: TeamTask - система управления задачами для малой команды.

Цель: упростить постановку, распределение и контроль задач внутри малой команды, чтобы участники видели свои задачи и сроки, а руководитель понимал общий статус работы.

Целевая аудитория: участники малой команды, тимлид или руководитель команды, заказчик (например, руководитель небольшого отдела).

Ключевые функции:

1. Регистрация и вход пользователей.
2. Создание, редактирование и удаление задач.
3. Назначение исполнителя, срока и приоритета задачи.
4. Изменение статуса задачи: «новая», «в работе», «на проверке», «выполнена».
5. Комментарии к задачам.
6. Поиск и фильтрация задач по исполнителю, статусу и сроку.
7. Уведомления о приближении дедлайна.
8. Простой отчёт по задачам для руководителя команды.

Участники проекта:

- заказчик - руководитель малой команды;
- руководитель проекта / тимлид;
- аналитик;
- дизайнер;
- frontend-разработчик;
- backend-разработчик (Python);
- тестировщик;
- DevOps-специалист (в малой команде может совмещаться с backend-разработчиком).

3.3. Перечень артефактов

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
1	Техническое задание	Документ	Фиксирует требования заказчика к системе	Аналитик	Вся команда, заказчик	Анализ требований	Word / PDF

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
2	План проекта	Документ	Описывает этапы, сроки и ответственных	Руководитель проекта	Команда, заказчик	Планирование	Excel / Word
3	Диаграмма вариантов использования	Схема	Показывает функции системы и роли пользователей	Аналитик	Команда, заказчик	Проектирование	draw.io / PDF
4	Макеты интерфейса	Макеты	Показывают внешний вид основных экранов	Дизайнер	Frontend-разработчик, заказчик	Проектирование	Figma
5	Схема базы данных	Схема	Описывает таблицы, поля и связи между ними	Backend-разработчик	Команда	Проектирование	draw.io / SQL
6	Спецификация API	Документ	Описывает запросы и ответы между frontend и backend	Backend-разработчик	Frontend-разработчик, тестировщик	Разработка	Markdown / OpenAPI
7	Исходный код на Python	Код	Реализует работу системы (backend на Python)	Backend-разработчик	Команда	Разработка	Git-репозиторий
8	Конфигурационные файлы	Конфигурация	Хранят настройки приложения и окружения (settings.py, .env, requirements.txt)	Backend-разработчик / DevOps	Команда, DevOps	Разработка / внедрение	Git-репозиторий
9	Тест-план	Документ	Определяет, что и как будет тестироваться	Тестировщик	Команда	Тестирование	Word / Confluence
10	Тест-кейсы	Тест-кейс	Содержат проверки функций системы (в том числе автотесты на pytest)	Тестировщик	Тестировщик, команда	Тестирование	Excel / TestIT / код в репозитории

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
11	Отчёт о тестировании	Документ	Фиксирует результаты тестов и найденные дефекты	Тестировщик	Команда, заказчик	Тестирование	Word / PDF
12	Руководство пользователя	Документ	Объясняет пользователям, как работать с системой	Аналитик / тестировщик	Пользователи	Внедрение	Word / PDF
13	Инструкция по развёртыванию	Документ	Описывает, как установить и запустить систему (в том числе установка зависимостей Python, запуск через pip и виртуальное окружение)	DevOps / backend-разработчик	DevOps, команда	Внедрение	Markdown
14	Журнал изменений	Документ	Фиксирует изменения по версиям системы	DevOps / backend-разработчик	Команда, пользователи	Сопровождение	Markdown в репозитории

3.4. Перечень протоколов

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
1	Протокол совещания	Фиксация обсуждаемых вопросов и принятых решений	Вся команда, заказчик	Еженедельно	Документ в Confluence / Notion
2	Протокол согласования требований	Определяет, как заказчик утверждает требования и изменения к ним	Аналитик, заказчик, руководитель проекта	По событию / разово	Документ, письмо
3	Протокол код-ревью	Определяет правила проверки кода перед слиянием в основную ветку (в том числе соблюдение PEP 8 для Python)	Разработчики	По каждому pull request	Запись в GitHub / GitLab

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
4	Протокол тестирования	Определяет, как фиксируются результаты тестов и дефекты (включая запуск pytest)	Тестировщик, разработчики	По событию	Отчёт, запись в трекере
5	Протокол приёмки	Определяет, как заказчик принимает готовый результат	Заказчик, руководитель проекта, тестировщик	Разово / по этапам	Акт / документ
6	Протокол управления изменениями	Определяет, как вносятся изменения в требования и задачи	Аналитик, заказчик, руководитель проекта	По событию	Заявка / документ

3.5. Связь артефактов и протоколов

1. **Протокол согласования требований связан с техническим заданием.** Он регулирует, как заказчик утверждает ТЗ. Без этого протокола требования могут изменить без ведома заказчика, и команда сделает не то, что нужно.
2. **Протокол код-ревью связан с исходным кодом на Python.** По этому протоколу разработчики проверяют код перед слиянием, в том числе смотрят, чтобы код соответствовал PEP 8 и не ломал существующие тесты. Если протокола нет, в основную ветку может попасть непонятный или плохо работающий код.
3. **Протокол тестирования связан с тест-кейсами и отчётом о тестировании.** По нему фиксируется, какие тесты прошли (например, автотесты на pytest), какие дефекты найдены и кто за них отвечает. Без этого результаты тестирования будут разрозненными.
4. **Протокол приёмки связан с отчётом о тестировании и руководством пользователя.** Заказчик принимает работу по результатам тестов и документации. Так обе стороны понимают, что именно было сдано.
5. **Протокол управления изменениями связан с техническим заданием и планом проекта.** Если появляется новое требование, оно проходит через этот протокол, а потом отражается в ТЗ и плане.

3.6. Обоснование выбора

Для любого проекта, на мой взгляд, обязательны техническое задание, исходный код, тест-кейсы и руководство пользователя. Без ТЗ непонятно, что делать. Без кода нет самого продукта. Без тестов нельзя понять, работает ли система. Без руководства пользователям будет сложно разобраться.

Специфичными для TeamTask являются макеты интерфейса, схема базы данных и спецификация API. Для системы управления задачами это важно, потому что есть frontend, backend на Python и база данных, и им нужно договариваться между собой. Также для Python-проекта важны конфигурационные файлы (requirements.txt, .env, settings.py), потому что от них зависит, запустится ли приложение на другом компьютере.

Критичные протоколы для проекта - это протокол согласования требований, код-ревью, тестирования и приёмки. Если не будет протокола согласования требований, команда и заказчик могут по-разному понимать задачу. Если не будет код-ревью, качество кода упадёт (особенно если разработчики пишут на Python по-разному). Если не будет протокола тестирования, будет непонятно, какие дефекты уже найдены. Если не будет протокола приёмки, в конце проекта могут возникнуть споры о том, принята работа или нет.

Если какой-то артефакт или протокол будет отсутствовать, проект всё равно может продолжиться, но вырастет риск ошибок, недопонимания и лишней работы. Поэтому перечень артефактов и протоколов нужен не для галочки, а чтобы команда понимала, что и как она делает.